CO1 – Assignment 1

CSA0823 - Python Programming

Name: SURESH RAJ R

Reg.no: 1912260217

Task 1

Ans:

1. Inputs and Outputs

The milk payment system calculates the payment for farmers based on the quantity and quality of milk supplied.

Inputs:

- Farmer ID: Uniquely identifies each farmer.
- Litres: Quantity of milk supplied, which may contain decimal values.
- Fat percentage: Fat content of milk, which must be between 2.0% and 7.0%.
- SNF percentage: Solids-Not-Fat content of milk.
- Shift: Milk collection shift, such as Morning or Evening.

Outputs:

- Rate per litre: Calculated according to the milk quality and the specified payment formula.
- Amount: Payment for each entry, calculated as litres multiplied by the rate per litre.
- Ten-day total: Sum of all payment amounts for a farmer over ten days.

2. Constants

Constants are values that do not change during program execution.

- MIN_FAT = 2.0 – Minimum permitted fat percentage.
- MAX_FAT = 7.0 – Maximum permitted fat percentage.
- PERIOD_DAYS = 10 – Payment period of ten days.

Using named constants improves readability and simplifies maintenance.

3. Data Types

Data	Data Type	Reason
Farmer ID	String	Can contain letters or leading zeros.
Litres	Float	Milk quantity may contain decimals.
Fat percentage	Float	May contain decimal values.
SNF percentage	Float	May contain decimal values.
Shift	String	Represents Morning or Evening.
Rate per litre	Float	Represents the payment rate.
Amount	Float	Represents the payment for an entry.
Ten-day total	Float	Represents the total payment.

4. Validation Rules

1. Farmer ID must not be empty.
2. Milk quantity must not be negative.
3. Fat percentage must be between 2.0% and 7.0%, inclusive.
4. SNF percentage must be non-negative and within the applicable permitted range.
5. Shift must be Morning or Evening.
6. Payment amount must equal litres multiplied by the rate per litre.
7. The ten-day total must equal the sum of all valid payments for the farmer.

5. Assumptions

- Each farmer has a unique ID.
- Milk quantity and quality percentages may contain decimal values.
- A farmer may supply milk multiple times during the ten-day period.
- The rate per litre is determined using a predefined milk-quality formula.
- All valid entries for a farmer within the ten-day period contribute to the total.

6. Alternative Design and Complexity Analysis

Separate variables: Creating variables such as litres1, litres2, and litres3 for 300 farmers requires many variables and repetitive code, making the program difficult to maintain.

List of records: Storing each entry in a list allows the program to process all entries using loops. New entries can be added without creating new variable names.

- Time complexity: Processing $n$entries takes $O(n)$time.

- Space complexity: Storing $n$entries requires $O(n)$space.

Both approaches have the same asymptotic complexity when implemented appropriately. However, a list of records is more scalable, maintainable, and convenient for processing large numbers of farmers.

Program:

1. Constants

MIN_FAT = 2.0

MAX_FAT = 7.0

PERIOD_DAYS = 10

DEFAULT_RATE = 40.0

2. Store farmer records

farmers = []

3. Get the number of farmers

n = int(input("Enter the number of farmers: "))

4. Collect and validate data

```
for i in range(n):
    print(f"\nEnter details for farmer {i + 1}")

    farmer_id = input("Farmer ID: ").strip()
    litres = float(input("Litres of milk: "))
    fat = float(input("Fat percentage: "))
    snf = float(input("SNF percentage: "))
    shift = input("Shift (Morning/Evening): ").strip().title()

    Validate inputs
    if not farmer_id:
        print("Invalid Farmer ID")
```

```
    continue

if litres < 0:
    print("Milk quantity cannot be negative")
    continue

if not MIN_FAT <= fat <= MAX_FAT:
    print("Fat must be between 2% and 7%")
    continue

if snf < 0:
    print("SNF cannot be negative")
    continue

if shift not in ("Morning", "Evening"):
    print("Invalid shift")
    continue
```

5. Calculate payment

Replace DEFAULT_RATE with the actual rate formula specified in the assignment.

```
rate = DEFAULT_RATE
amount = litres * rate
```

6. Store the valid record

```
farmers.append({
    "farmer_id": farmer_id,
    "litres": litres,
    "fat": fat,
    "snf": snf,
```

```
        "shift": shift,
        "rate": rate,
        "amount": amount
    })

7. Display individual payment details
print("\n========== MILK PAYMENT REPORT ==========")

for farmer in farmers:
    print("\nFarmer ID:", farmer["farmer_id"])
    print("Litres:", farmer["litres"])
    print("Fat %:", farmer["fat"])
    print("SNF %:", farmer["snf"])
    print("Shift:", farmer["shift"])
    print("Rate per litre: Rs.", farmer["rate"])
    print("Payment amount: Rs.", farmer["amount"])

8. Calculate ten-day totals for each farmer
totals = {}

for farmer in farmers:
    fid = farmer["farmer_id"]

    if fid not in totals:
        totals[fid] = 0.0

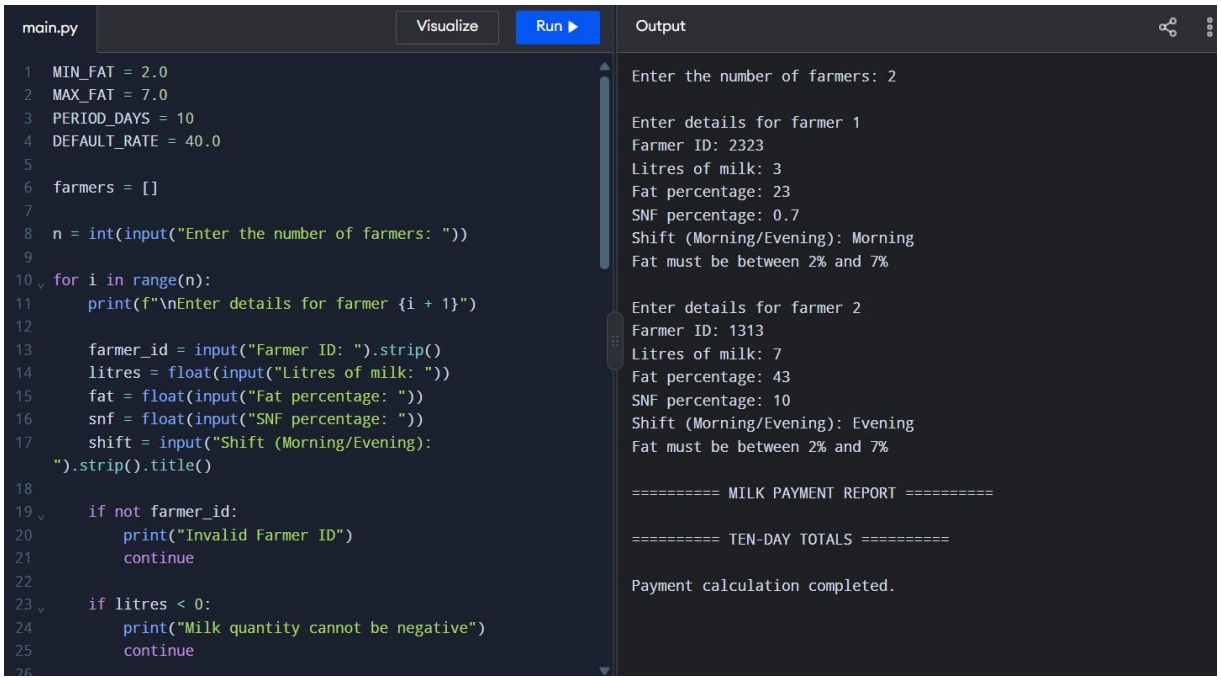
    totals[fid] += farmer["amount"]

9. Display ten-day totals
print("\n========== TEN-DAY TOTALS ==========")
```

```
for fid, total in totals.items():
    print("Farmer ID:", fid)
    print("Total payment for", PERIOD_DAYS,
          "days: Rs.", round(total, 2))

print("\nPayment calculation completed.")
```

OUTPUT



Task 2

Ans:

1. Types of Values Using type()

Python provides the type() function to identify the data type of a value.

Value	Data Type	Explanation
10	int	Whole number
10.0	float	Decimal number
"10"	str	Text enclosed in quotes
True	bool	Boolean value
[10, 8.5]	list	Collection of values

2. Division Operators

- 7 / 2 = 3.5: Normal division returns a float.
- 7 // 2 = 3: Floor division returns the floor of the quotient.
- 7 % 2 = 1: Modulus returns the remainder.
- -7 // 2 = -4: Floor division rounds down towards negative infinity, not towards zero.

3. Floating-Point Precision

The expression 0.1 + 0.2 == 0.3 returns False because decimal fractions such as 0.1 and 0.2 cannot be represented exactly in binary floating-point format.

Therefore, the calculated result is approximately 0.30000000000000004, rather than exactly 0.3.

4. Why Money Should Not Use Ordinary Floats

Floating-point calculations can introduce small precision errors, which may cause inaccuracies in financial totals.

Alternatives:

- Integer paise: Store money as whole paise. For example, ₹33.40 becomes 3340 paise. This provides exact arithmetic for amounts expressed in whole paise.
- round(): Round the result to two decimal places when appropriate. However, rounding alone does not guarantee exact financial arithmetic.
- Decimal: Use Python's decimal module to perform accurate decimal arithmetic.

The Decimal type is suitable for financial calculations because it can represent decimal amounts such as ₹33.40 exactly.

PROGRAM:

```
from decimal import Decimal

import sys

import time


print("DATA TYPES")

print(type(10))

print(type(10.0))

print(type("10"))

print(type(True))

print(type([10, 8.5]))
```

```
print("\nDIVISION OPERATORS")
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)

print("\nFLOATING-POINT PRECISION")
print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)
print("Decimal comparison:",
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

def total_decimal_float(items):
    total = Decimal("0.00")
    for amount in items:
```

```
        total += Decimal(float(amount))
    return total

def total_paise(items):
    total = 0
    for amount in items:
        rupees, paise = amount.split(".")
        total += int(rupees) * 100 + int(paise.ljust(2, "0"))
    return total

def total_rounded_float(items):
    total = 0.0
    for amount in items:
        total = round(total + float(amount), 2)
    return total

print("\nPAYMENT TOTALS")
print("Float:", total_float(payments))
print("Decimal:", total_decimal(payments))
print("Decimal(float):", total_decimal_float(payments))
print("Integer paise:", total_paise(payments), "paise")
print("Integer paise in rupees:", total_paise(payments) / 100)
print("Rounded float:", total_rounded_float(payments))
print("Expected total:", Decimal("33400.30"))

print("\nMEMORY USAGE")
print("Float size:", sys.getsizeof(33.40), "bytes")
print("Decimal size:", sys.getsizeof(Decimal("33.40")), "bytes")
print("Integer size:", sys.getsizeof(3340), "bytes")
```

```
print("\nTIME COMPLEXITY")
print("Each method processes n payments in O(n) time.")
print("Each method uses O(1) additional space for the running total.")
```

OUTPUT

```
= RESTART: C:/Users/Purushothaman N/AppData/Local/Programs/Python/P
DATA TYPES
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>

DIVISION OPERATORS
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
-7 // 2 = -4

FLOATING-POINT PRECISION
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False
Decimal comparison: True

PAYMENT TOTALS
Float: 33400.30000000061
Decimal: 33400.30
Decimal(float): 33400.299999999998578931182 78
Integer paise: 3340030 paise
Integer paise in rupees: 33400.3
Rounded float: 33400.3
Expected total: 33400.30

MEMORY USAGE
Float size: 24 bytes
Decimal size: 120 bytes
Integer size: 28 bytes

TIME COMPLEXITY
Each method processes n payments in O(n) time.
Each method uses O(1) additional space for the running total.
Each method uses O(1) additional space for the running total.
```

TASK 3

Answer:

1. Naming Rules and Conventions

Python variables and constants should have meaningful names.

- Variable names should begin with a letter or underscore.
- Variable names must not contain spaces or begin with a number.

- Python is case-sensitive; rate and Rate are different variables.
- Variables use snake_case, such as farmer_id and milk_quantity.
- Constants use UPPER_CASE, such as BASE_RATE and MIN_FAT.

Meaningful names improve readability and make the program easier to maintain.

2. Named Constants

BASE_RATE = 30.0

FAT_RATE = 0.40

SNF_RATE = 0.30

MIN_FAT = 2.0

MAX_FAT = 7.0

PERIOD_DAYS = 10

These constants represent the base rate, quality-based rate factors, permitted fat limits, and payment period. If the rates change, they can be updated in one place.

Python does not have a built-in declaration that makes an ordinary variable permanently constant. Uppercase names are a convention indicating that a value should not be changed.

3. Dynamic Typing

Python is dynamically typed, meaning a variable can refer to values of different types during execution.

value = 10

print(type(value))

value = "Milk"

print(type(value))

The first output identifies value as an integer (int), while the second identifies it as a string (str). Python allows this reassignment without explicitly declaring the variable's type.

4. Multiple Assignment and Swapping

Python supports assigning values to multiple variables in one statement.

a, b = 10, 20

a, b = b, a

print(a, b)

Output:

20 10

Multiple assignment makes code shorter, and swapping exchanges two values without requiring a temporary variable.

5. Alternative Concept and Complexity Analysis

Magic numbers: If rate values are written directly inside several expressions, every occurrence must be located and changed when rates are revised. If a value appears in *k*places, up to *k*edits may be needed.

Named constants: Defining rates once means a revision usually requires changing only one definition.

Dictionary approach:

```
RATE_CARD = {
    "base": 30.0,
    "fat": 0.40,
    "snf": 0.30
}
```

A dictionary groups related rates together and provides convenient access using keys.

Complexity analysis:

- Time: Changing constants does not change the program's overall running-time complexity.
- Space: Named constants and dictionaries require memory, but the difference is negligible for a small set of rates.
- Maintainability: Named constants and dictionaries are easier to update than repeated magic numbers.

Program:

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0
PERIOD_DAYS = 10


farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter litres of milk: "))
```

```
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))

if not farmer_id.strip():
    print("Invalid Farmer ID")
elif litres < 0:
    print("Milk quantity cannot be negative")
elif not MIN_FAT <= fat <= MAX_FAT:
    print("Fat must be between 2% and 7%")
elif snf < 0:
    print("SNF cannot be negative")
else:
    rate = BASE_RATE + (fat * FAT_RATE) + (snf * SNF_RATE)
    amount = litres * rate

    print("Farmer ID:", farmer_id)
    print("Rate per litre:", round(rate, 2))
    print("Payment amount:", round(amount, 2))

a, b = 10, 20
print("Before swapping:", a, b)

a, b = b, a
print("After swapping:", a, b)

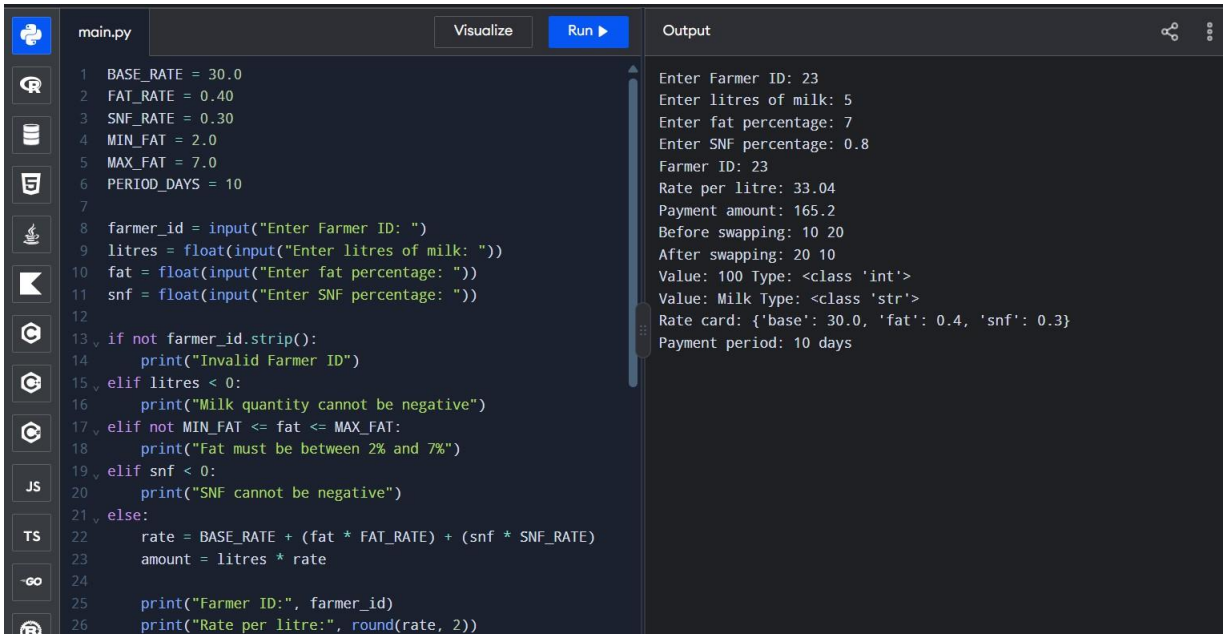
value = 100
print("Value:", value, "Type:", type(value))

value = "Milk"
print("Value:", value, "Type:", type(value))
```

```
RATE_CARD = {
    "base": 30.0,
    "fat": 0.40,
    "snf": 0.30
}

print("Rate card:", RATE_CARD)
print("Payment period:", PERIOD_DAYS, "days")
```

OUTPUT



TASK 4

1. Expressions for Fat Steps, SNF Steps, Rate and Amount

Python expressions calculate the number of quality steps, the rate per litre, and the payment amount.

fat_steps = round((fat - 3.5) / 0.1)

snf_steps = round((snf - 8.0) / 0.1)

rate = BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE

amount = litres * rate

Here, 3.5 and 8.0 are assumed reference values, and the rate factors are assumed constants. The actual values must match the assignment's payment rules.

2. Operator Precedence and Associativity

Python follows operator precedence when evaluating expressions.

- ** performs exponentiation before most arithmetic operators.
- *, /, //, and % have higher precedence than + and -.
- Parentheses () can explicitly control evaluation order.
- Exponentiation is right-associative: 2 ** 3 ** 2 gives 512.
- Unary minus has lower precedence than exponentiation: -2 ** 2 gives -4, whereas (-2) ** 2 gives 4.

3. Floating-Point Precision and round()

The expression:

(4.2 - 3.5) / 0.1

may produce 7.000000000000001 because decimal fractions cannot always be represented exactly in binary floating-point arithmetic.

Using round() helps obtain the intended step count:

round((4.2 - 3.5) / 0.1)

Output:

7

4. Difference Between round(), int() and math.floor()

Expression	Result	Explanation
round(3.7)	4	Rounds to the nearest integer.
int(3.7)	3	Truncates the fractional part towards zero.
math.floor(3.7)	3	Rounds down towards negative infinity.
round(-3.7)	-4	Rounds to the nearest integer.
int(-3.7)	-3	Truncates towards zero.
math.floor(-3.7)	-4	Rounds down towards negative infinity.

Alternative concept: For fat values of 4.2 and 3.1, the step counts relative to 3.5 are calculated as follows.

Method	Fat = 4.2	Fat = 3.1
round()	7	-4
int()	7	-3

Method	Fat = 4.2	Fat = 3.1
math.floor()	7	-4

If the rate is calculated using BASE_RATE + steps * FAT_RATE, the resulting rates will differ when the step counts differ.

All three methods require **O(1) time and O(1) additional space** for a single calculation.

5. Comparison Chaining and Boolean Operators

Python supports comparison chaining:

2.0 <= fat <= 7.0

This checks whether fat is between 2.0 and 7.0, inclusive. It is equivalent to:

fat >= 2.0 and fat <= 7.0

Boolean operators include:

- and: True when both conditions are true.
- or: True when at least one condition is true.
- not: Reverses a Boolean value.

6. Three Unexpected Results

Example 1:

0.1 + 0.2

Output:

0.30000000000000004

Explanation: Binary floating-point cannot represent these decimal fractions exactly.

Example 2:

-7 // 2

Output:

-4

Explanation: Floor division rounds down towards negative infinity rather than towards zero.

Example 3:

-2 ** 2

Output:

-4

Explanation: Exponentiation occurs before unary minus, so the expression is interpreted as -(2 ** 2).

Program:

```
import math

BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0

litres = float(input("Enter litres of milk: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))

if litres < 0:
    print("Invalid milk quantity")

elif not MIN_FAT <= fat <= MAX_FAT:
    print("Invalid fat percentage")

elif snf < 0:
    print("Invalid SNF percentage")

else:
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.0) / 0.1)

    rate = BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE
    amount = litres * rate

    print("Fat steps:", fat_steps)
```

```
    print("SNF steps:", snf_steps)
    print("Rate per litre:", round(rate, 2))
    print("Payment amount:", round(amount, 2))

print("\nOperator Expressions")
print("(4.2 - 3.5) / 0.1 =", (4.2 - 3.5) / 0.1)
print("Rounded steps:", round((4.2 - 3.5) / 0.1))
print("0.1 + 0.2 =", 0.1 + 0.2)
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)
print("-2 ** 2 =", -2 ** 2)
print("(-2) ** 2 =", (-2) ** 2)

print("\nRounding Comparison")
for value in [4.2, 3.1]:
    steps = (value - 3.5) / 0.1
    print("Fat:", value)
    print("round():", round(steps))
    print("int():", int(steps))
    print("floor():", math.floor(steps))

print("\nComparison Chaining")
print("Fat valid:", 2.0 <= fat <= 7.0)
```

OUTPUT

Output

```
Enter litres of milk: 10
Enter fat percentage: 2.3
Enter SNF percentage: 1.54
Fat steps: -12
SNF steps: -65
Rate per litre: 5.7
Payment amount: 57.0

Operator Expressions
(4.2 - 3.5) / 0.1 = 7.000000000000002
Rounded steps: 7
0.1 + 0.2 = 0.30000000000000004
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
-7 // 2 = -4
-2 ** 2 = -4
(-2) ** 2 = 4

Rounding Comparison
Fat: 4.2
round(): 7
int(): 7
floor(): 7
Fat: 3.1
round(): -4
```

```
Rounding Comparison
Fat: 4.2
round(): 7
int(): 7
floor(): 7
Fat: 3.1
round(): -4
int(): -3
floor(): -4

Comparison Chaining
Fat valid: True
```

Task 5

Ans:

1. Input and Type Conversion

Python's input() always returns a string. We use int() for whole numbers and float() for decimal numbers.

2. Invalid Input Handling

- "8.5" can be converted to float.
- "eight" causes a ValueError because it is not a valid number.
- "8,5" causes a ValueError because Python uses a decimal point, not a comma.

We can handle invalid inputs using try-except ValueError.

3. Formatted Output

F-strings display results neatly. For example, f"{amount:8.2f}" displays a number using a width of 8 characters and 2 decimal places.

4. Alternative Approach and Complexity

String validation checks input before conversion, while try-except attempts conversion and catches errors. Both take approximately O(1) time per entry for ordinary short inputs. try-except handles different numeric formats more reliably.

Program:

```
MIN_FAT = 2.0

MAX_FAT = 7.0

RATE = 40.0

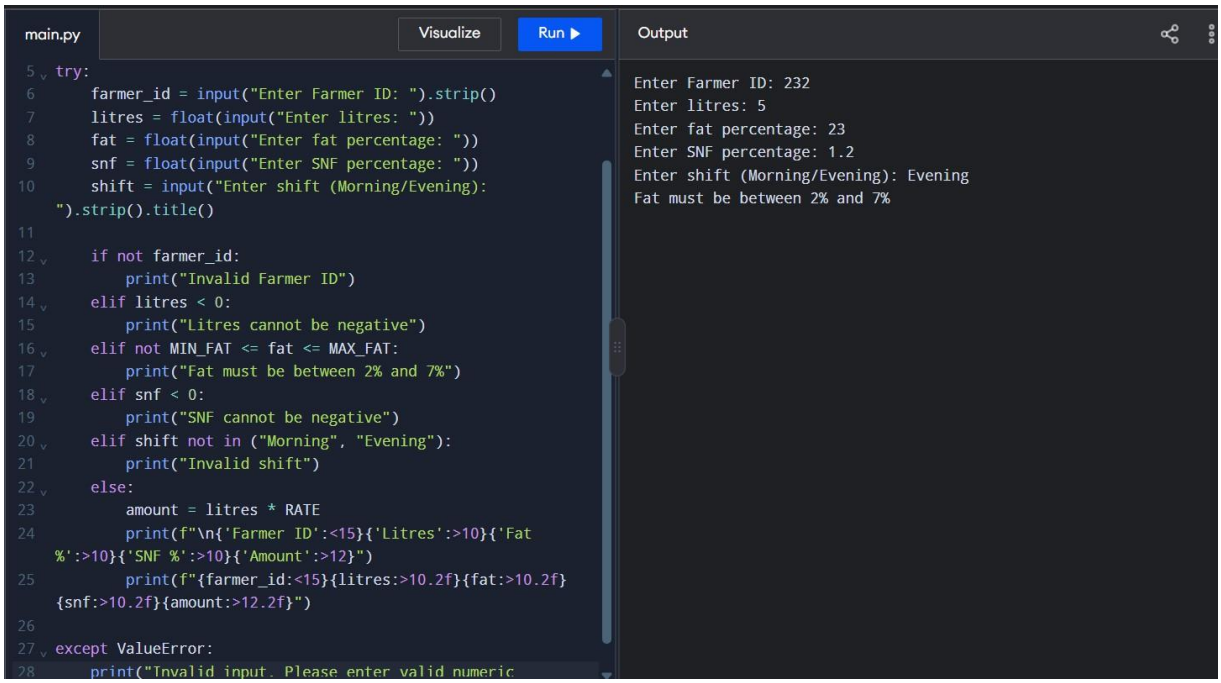

try:
    farmer_id = input("Enter Farmer ID: ").strip()
    litres = float(input("Enter litres: "))
    fat = float(input("Enter fat percentage: "))
    snf = float(input("Enter SNF percentage: "))
    shift = input("Enter shift (Morning/Evening): ").strip().title()

    if not farmer_id:
        print("Invalid Farmer ID")
    elif litres < 0:
```

```
        print("Litres cannot be negative")
    elif not MIN_FAT <= fat <= MAX_FAT:
        print("Fat must be between 2% and 7%")
    elif snf < 0:
        print("SNF cannot be negative")
    elif shift not in ("Morning", "Evening"):
        print("Invalid shift")
    else:
        amount = litres * RATE
        print(f"\n{'Farmer ID':<15}{'Litres':>10}{'Fat %':>10}{'SNF %':>10}{'Amount':>12}")
        print(f"{farmer_id:<15}{litres:>10.2f}{fat:>10.2f}{snf:>10.2f}{amount:>12.2f}")

except ValueError:
    print("Invalid input. Please enter valid numeric values.")
```

OUTPUT



TASK 6

Ans.

1. Functions Required

- rate_per_litre() – Accepts fat and SNF percentages and returns the rate per litre.

- payout() – Accepts litres and rate and returns the payment amount.
- ten_day_total() – Accepts a list of payments and returns their total.

2. Returning vs Printing

return sends a result back to the calling code so it can be reused in calculations. print() only displays the result on the screen.

3. Local and Global Variables

Global constants can be read inside functions. Local variables exist only within their function. Constants should not be modified inside functions because this makes the program harder to maintain.

4. Docstrings

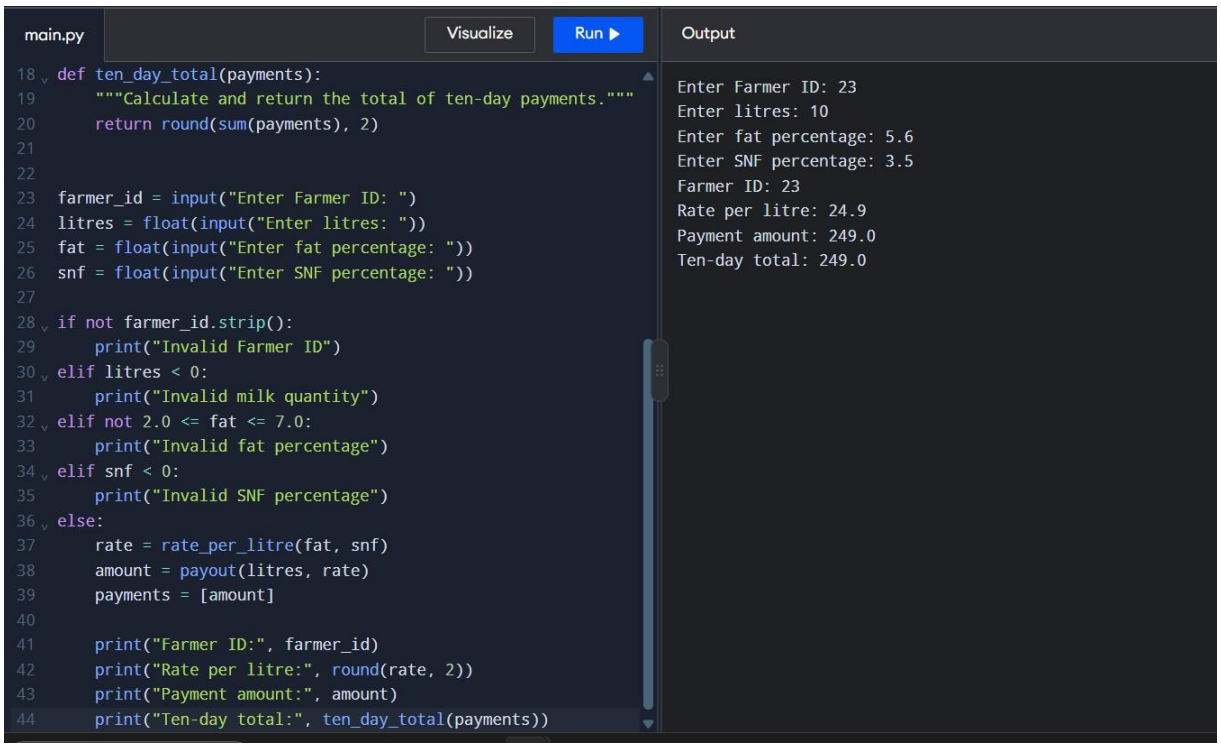
Docstrings describe a function's purpose, parameters, and return value. They are written using triple quotes ("""...""").

5. Alternative Concept and Complexity

Writing everything in one script increases code repetition and makes maintenance difficult. Functions improve reuse and readability. Lambda expressions are suitable for simple operations but are less appropriate for complex calculations.

For *n*entries, processing takes O(n) time and storing the entries requires O(n) space. Function-call overhead is generally small and does not change the overall time complexity.

Program:



Task 7

Ans.

1. Importing Modules

Python supports three ways to import a module:

- import milkrate – Access functions using milkrate.payout().
- from milkrate import payout – Access payout() directly.
- import milkrate as mr – Access functions using mr.payout().

2. Purpose of if __name__== "__main__":

This condition executes the test code only when the file runs directly. When another script imports the module, the test code does not execute, preventing unwanted output.

3. Standard Modules

Python provides built-in modules such as math for mathematical operations and decimal for accurate decimal arithmetic.

4. Module Search Path

Python searches for modules in the current directory and the directories listed in sys.path.

5. Alternative Concept and Complexity

Copying functions into multiple scripts causes code duplication. Whenever rates change, every copy must be updated. Using a module allows the rate to be changed in one place.

Importing a module has a one-time loading cost. Python caches imported modules in sys.modules, avoiding repeated loading within the same process. The calculation remains **O(1)** per payment.

Program:

```
"""Rate card of the village milk co-operative."""


BASE_RATE = 30.00

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30



def rate_per_litre(fat, snf):

    """Return the price per litre, rounded to two decimals."""

    fat_steps = round((fat - 3.5) / 0.1)

    snf_steps = round((snf - 8.5) / 0.1)
```

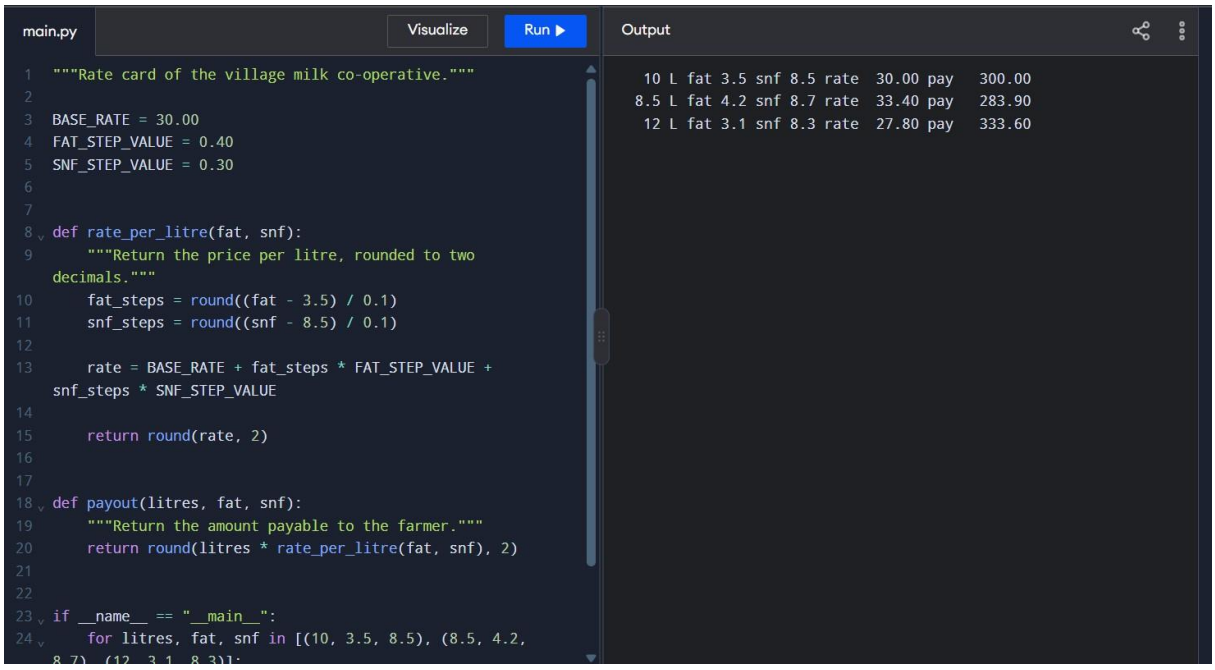
```
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE

    return round(rate, 2)


def payout(litres, fat, snf):
    """Return the amount payable to the farmer."""
    return round(litres * rate_per_litre(fat, snf), 2)


if __name__== "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Output:



Task 8:

Ans

1. Steps of Algorithmic Problem Solving

1. **Understand:** Identify the inputs, outputs, and payment rules.
2. **Plan:** Prepare pseudocode and a flowchart.
3. **Write:** Convert the algorithm into Python code.
4. **Test:** Check valid inputs, invalid inputs, payment calculations, and ten-day totals.

Pseudocode:

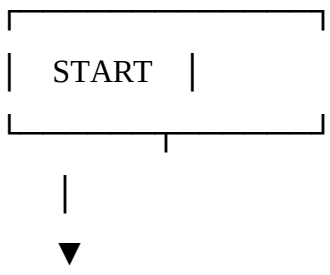
```
BEGIN
   SET total = 0

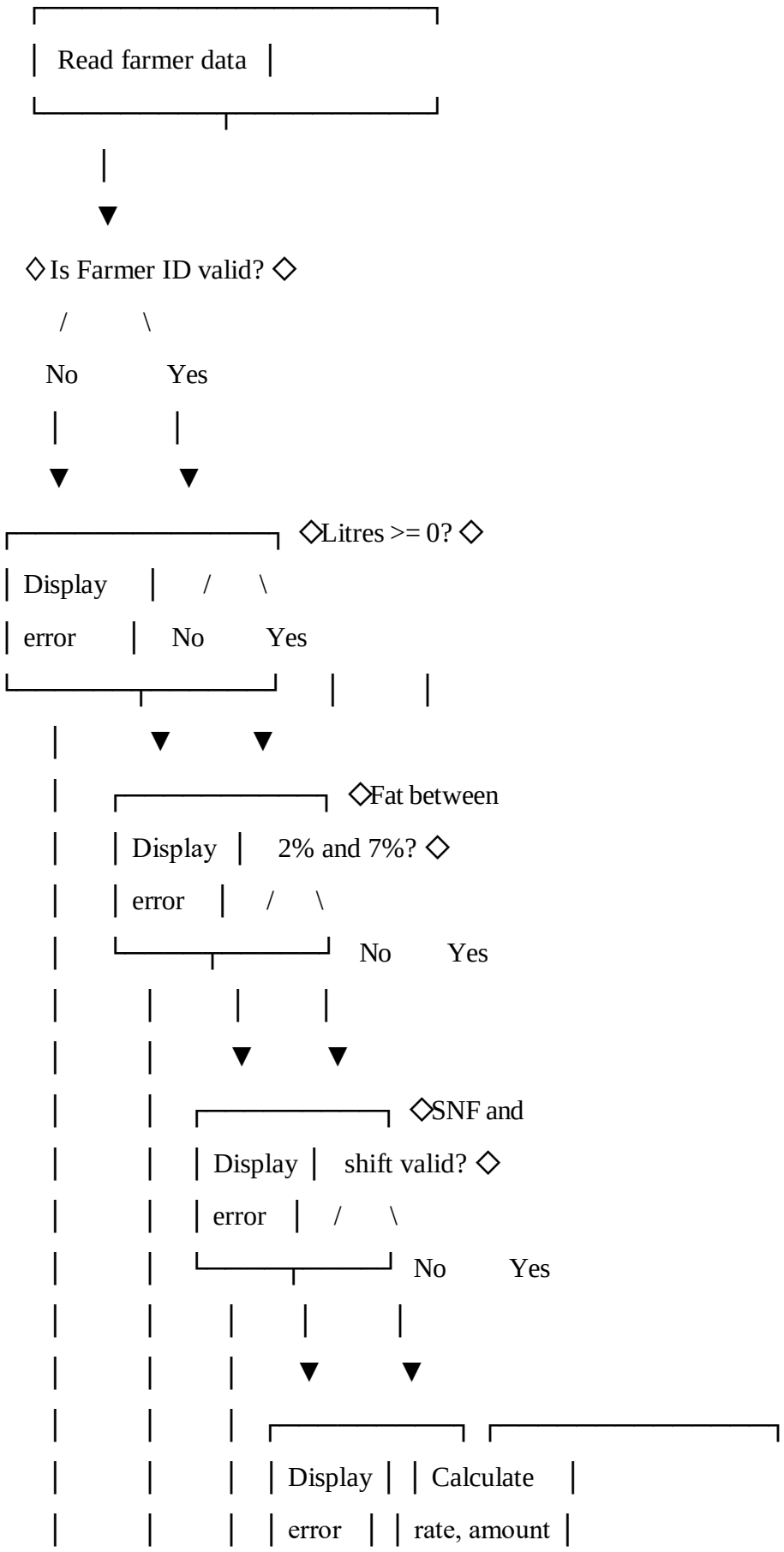
   FOR each of the 10 days
      INPUT farmer_id, litres, fat, snf, shift

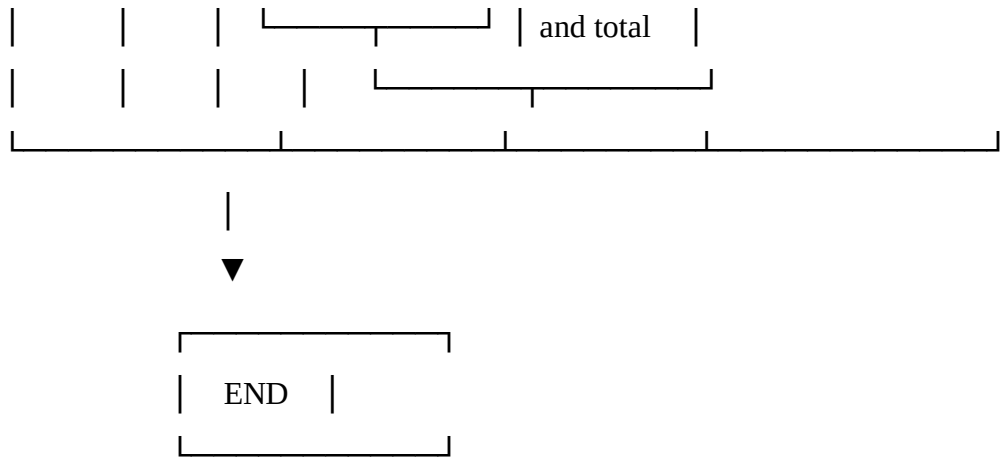
      IF inputs are invalid THEN
         DISPLAY error message
      ELSE
         CALCULATE rate using fat and SNF
         CALCULATE amount = litres * rate
         total = total + amount
         DISPLAY amount
      END IF
   END FOR

   DISPLAY ten-day total
END
```

Flowchart:




Read farmer data
Is Farmer ID valid?
No
Yes
Display error
Litres >= 0?
Fat between 2% and 7%?
SNF and shift valid?
Display error
Calculate rate, amount



4. Hand Trace for Three Entries

Assume the rate is calculated using:

BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, and SNF_STEP_VALUE = 0.30.

Entry	Litres	Fat	SNF	Rate (₹)	Amount (₹)	Running Total (₹)
1	10	3.5	8.5	30.00	300.00	300.00
2	8.5	4.2	8.7	33.40	283.90	583.90
3	12	3.1	8.3	27.80	333.60	917.50

Final total for the three entries = ₹917.50.

5. Alternative Concept and Complexity Analysis

Approach 1: Store all entries in a list

- Time complexity: $O(n)$.
- Space complexity: $O(n)$.
- Advantage: Records can be reviewed, corrected, or processed later.

Approach 2: Maintain a running total

- Time complexity: $O(n)$.
- Additional space complexity: $O(1)$.
- Advantage: Uses less memory because only the current total needs to be maintained.

Storing entries is necessary when records must be audited, displayed again, or used for further calculations. A running total is preferable when only the final payment total is required.

Program:

BASE_RATE = 30.00

```
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)


def payout(litres, fat, snf):
    return round(litres * rate_per_litre(fat, snf), 2)


farmer_id = input("Enter Farmer ID: ").strip()
total = 0.0
entries = 0

for day in range(1, 11):
    print(f"\nDay {day}")

    try:
        litres = float(input("Enter litres: "))
        fat = float(input("Enter fat percentage: "))
        snf = float(input("Enter SNF percentage: "))
        shift = input("Enter shift (Morning/Evening): ").strip().title()

        if not farmer_id:
            print("Invalid Farmer ID")
```

```
            break
        elif litres < 0:
            print("Milk quantity cannot be negative")
            continue
        elif not 2.0 <= fat <= 7.0:
            print("Fat must be between 2% and 7%")
            continue
        elif snf < 0:
            print("SNF cannot be negative")
            continue
        elif shift not in ("Morning", "Evening"):
            print("Invalid shift")
            continue

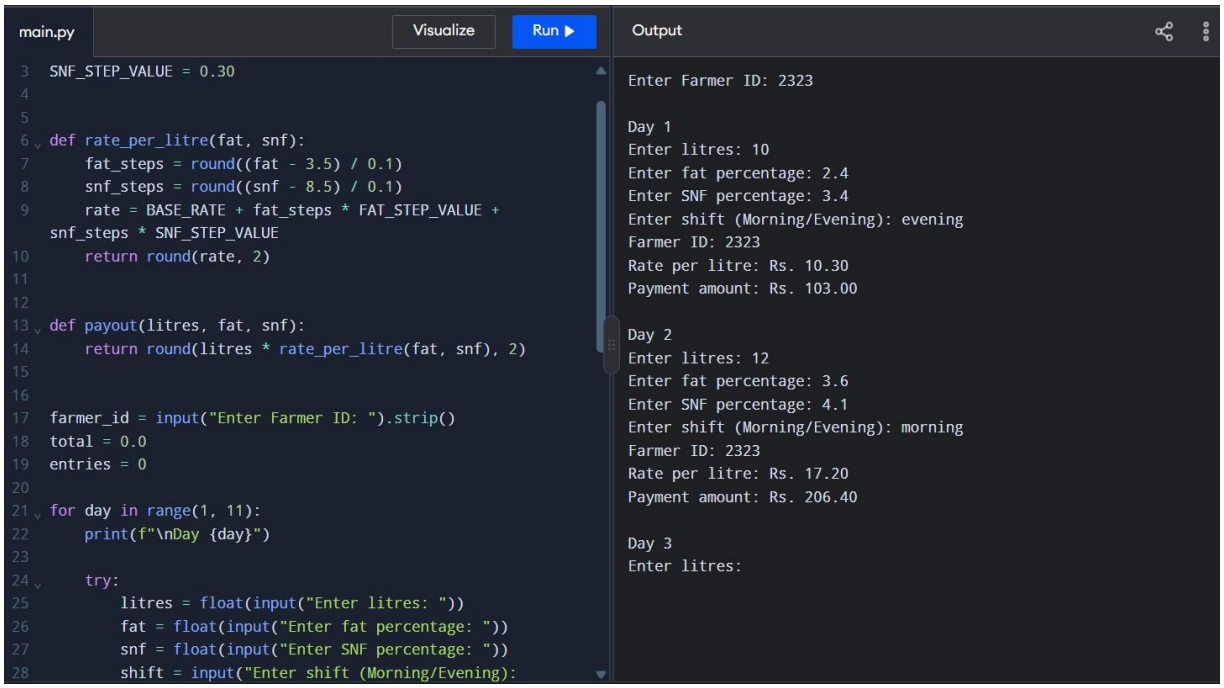
        rate = rate_per_litre(fat, snf)
        amount = payout(litres, fat, snf)
        total += amount
        entries += 1

        print(f"Farmer ID: {farmer_id}")
        print(f"Rate per litre: Rs. {rate:.2f}")
        print(f"Payment amount: Rs. {amount:.2f}")

    except ValueError:
        print("Invalid input. Please enter numeric values correctly.")

print(f"\nValid entries: {entries}")
print(f"Ten-day total: Rs. {total:.2f}")
```

OUTPUT:



TASK 9:

Ans

Program

BASE_RATE = 30.00

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30

MIN_FAT = 2.0

MAX_FAT = 7.0

```
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)
```

```
def payout(litres, fat, snf):
    return round(litres * rate_per_litre(fat, snf), 2)
```

```
def test_calculator():
    tests = [
        (10, 3.5, 8.5, 30.00, 300.00),
        (10, 4.0, 9.0, 33.50, 335.00),
        (10, 3.0, 8.0, 26.50, 265.00),
        (10, 2.0, 8.5, 24.00, 240.00),
        (10, 7.0, 8.5, 44.00, 440.00)
    ]

    print("LITRES FAT SNF EXPECTED_RATE ACTUAL_RATE EXPECTED_AMOUNT
ACTUAL_AMOUNT")

    for litres, fat, snf, expected_rate, expected_amount in tests:
        actual_rate = rate_per_litre(fat, snf)
        actual_amount = payout(litres, fat, snf)

        print(litres, fat, snf, expected_rate, actual_rate,
              expected_amount, actual_amount)

        assert actual_rate == expected_rate
        assert actual_amount == expected_amount

    assert rate_per_litre(3.5, 8.5) == 30.00

    for invalid_fat in [1.9, 7.1]:
        assert not MIN_FAT <= invalid_fat <= MAX_FAT

    print("\nAll tests passed!")
```

```
def main():
    print("1. Calculate Payment")
    print("2. Run Tests")

    choice = input("Enter choice: ")

    if choice == "1":
        try:
            farmer_id = input("Enter Farmer ID: ").strip()
            litres = float(input("Enter litres: "))
            fat = float(input("Enter fat percentage: "))
            snf = float(input("Enter SNF percentage: "))

            if not farmer_id:
                print("Invalid Farmer ID")
            elif litres < 0:
                print("Litres cannot be negative")
            elif not MIN_FAT <= fat <= MAX_FAT:
                print("Fat must be between 2% and 7%")
            elif snf < 0:
                print("Invalid SNF percentage")
            else:
                rate = rate_per_litre(fat, snf)
                amount = payout(litres, fat, snf)

                print(f"Farmer ID: {farmer_id}")
                print(f"Rate per litre: Rs. {rate:.2f}")
                print(f"Payment amount: Rs. {amount:.2f}")
```

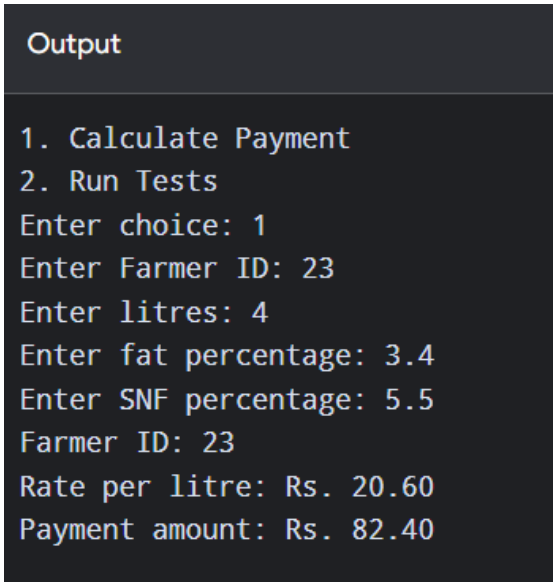
```
        except ValueError:
            print("Invalid input. Enter numeric values correctly.")

    elif choice == "2":
        test_calculator()

    else:
        print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:



```
Output

1. Calculate Payment
2. Run Tests
Enter choice: 2
LITRES FAT SNF EXPECTED_RATE ACTUAL_RATE EXPECTED_AMOUNT
ACTUAL_AMOUNT
10 3.5 8.5 30.0 30.0 300.0 300.0
10 4.0 9.0 33.5 33.5 335.0 335.0
10 3.0 8.0 26.5 26.5 265.0 265.0
10 2.0 8.5 24.0 24.0 240.0 240.0
10 7.0 8.5 44.0 44.0 440.0 440.0

All tests passed!
```

TASK 10:

Ans

1. Float vs Decimal

Floating-point numbers may introduce small precision errors, whereas Python's Decimal provides accurate decimal arithmetic when initialized from strings.

For 300, 6,000, and 150,000 payments, both methods should be tested to compare their totals and differences.

2. Execution Time

The time.perf_counter() function measures execution time.

- Float calculations are generally faster.
- Decimal calculations may take more time because they use decimal arithmetic.
- Both methods take **O(n) time** for *n*payments.

3. Rounding Methods

- **Round each payment:** Each payment is rounded to two decimal places before addition.
- **Round only the total:** All values are added first, and the final result is rounded to two decimal places.

For financial calculations, use Decimal with appropriate rounding rules to obtain reliable totals.

4. Alternative Concept: Rate Table

Instead of recalculating the rate for every entry, calculate it once for each unique fat and SNF combination and store it in a dictionary.

- Recalculating each rate takes O(1) time per entry.
- Dictionary lookup takes O(1) average time per entry.
- Storing *k*unique combinations requires O(k) additional space.

A rate table is especially useful when many payments share the same fat and SNF values.

Program:

```
from decimal import Decimal
from time import perf_counter

PAYMENT = "33.40"
SIZES = [300, 6000, 150000]



def float_total(n):
    total = 0.0
    for _ in range(n):
        total += float(PAYMENT)
    return total



def decimal_total(n):
    total = Decimal("0.00")
    amount = Decimal(PAYMENT)

    for _ in range(n):
        total += amount

    return total
```

```
def rounded_payment_total(n):
    total = Decimal("0.00")
    amount = Decimal(PAYMENT)

    for _ in range(n):
        total += amount.quantize(Decimal("0.01"))

    return total


print(f"{'Payments':>10} {'Float Total':>18} {'Decimal Total':>18} {'Difference':>15}")
for n in SIZES:
    start = perf_counter()
    f_total = float_total(n)
    float_time = perf_counter() - start

    start = perf_counter()
    d_total = decimal_total(n)
    decimal_time = perf_counter() - start

    difference = abs(Decimal(str(f_total)) - d_total)

    print(f"{n:10} {f_total:18.2f} {d_total:18.2f} {difference:15}")

    print(f"Float time: {float_time:.6f} seconds")
    print(f"Decimal time: {decimal_time:.6f} seconds")

    assert d_total == rounded_payment_total(n)

print("\nRate Table Example")
```

```
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(30.00 + fat_steps * 0.40 + snf_steps * 0.30, 2)



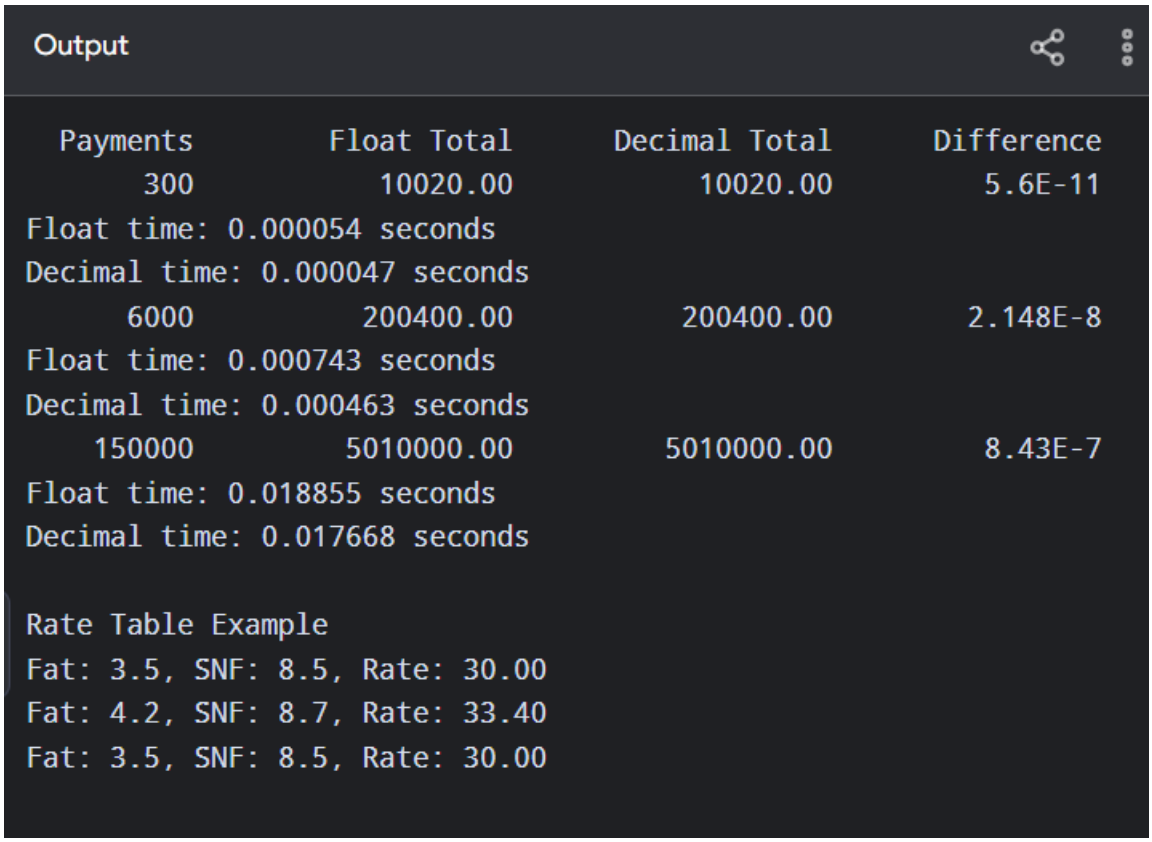
entries = [(3.5, 8.5), (4.2, 8.7), (3.5, 8.5)]

rate_table = {}

for fat, snf in entries:
    key = (fat, snf)
    if key not in rate_table:
        rate_table[key] = rate_per_litre(fat, snf)

for fat, snf in entries:
    print(f"Fat: {fat}, SNF: {snf}, Rate: {rate_table[(fat, snf)]:.2f}")
```

OUTPUT:



Task 11:

Ans

1. Recommended Data Types and Rounding

- Use str for farmer IDs and shift names.
- Use float for milk quantity and fat/SNF percentages.
- Use Decimal for payment rates and amounts to avoid floating-point precision errors.
- Round each payment to **two decimal places** using the co-operative's approved rounding rule.

2. Rate Revision

Store the base rate and fat/SNF step values as named constants in a separate milkrate.py module. When rates change, update the constants in one place rather than modifying every calculation.

For a larger system, store the rate card in a JSON file so authorized staff can revise rates without changing the program code.

3. Limitations

- Incorrect input data can produce incorrect payments.
- A simple program may not retain records permanently.

- Floating-point arithmetic can cause rounding errors if used for money.
- A local program does not automatically support multiple collection centres or simultaneous users.

4. District-Union Scale

The design is suitable for a small co-operative because functions, validation, and reusable modules make it easy to maintain. However, district-union deployment requires a database, authentication, backups, audit logs, and transaction management.

5. Alternative Concept: JSON Rate Card

A JSON file can store the rate values separately from the program.

Advantages:

- Rate revisions do not require modifying the calculation code.
- The same rate card can be shared across multiple programs.
- Configuration can be managed independently.

Complexity analysis:

- Reading $k$rate values takes $O(k)$time and $O(k)$space.
- Loading the file adds a small startup cost.
- Editing a JSON file avoids changing and redistributing the program for every rate revision.

Program:

```
import json

from decimal import Decimal, ROUND_HALF_UP


RATE_CARD = {
    "base_rate": "30.00",
    "fat_step_value": "0.40",
    "snf_step_value": "0.30",
    "min_fat": "2.0",
    "max_fat": "7.0"
}


BASE_RATE = Decimal(RATE_CARD["base_rate"])
FAT_STEP_VALUE = Decimal(RATE_CARD["fat_step_value"])
```

```
SNF_STEP_VALUE = Decimal(RATE_CARD["snf_step_value"])
MIN_FAT = Decimal(RATE_CARD["min_fat"])
MAX_FAT = Decimal(RATE_CARD["max_fat"])


def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (
        BASE_RATE
        + Decimal(fat_steps) * FAT_STEP_VALUE
        + Decimal(snf_steps) * SNF_STEP_VALUE
    )

    return rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)


def payout(litres, fat, snf):
    rate = rate_per_litre(float(fat), float(snf))
    amount = litres * rate
    return amount.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)


def main():
    farmer_id = input("Enter Farmer ID: ").strip()

    try:
        litres = Decimal(input("Enter litres: "))
        fat = Decimal(input("Enter fat percentage: "))
```

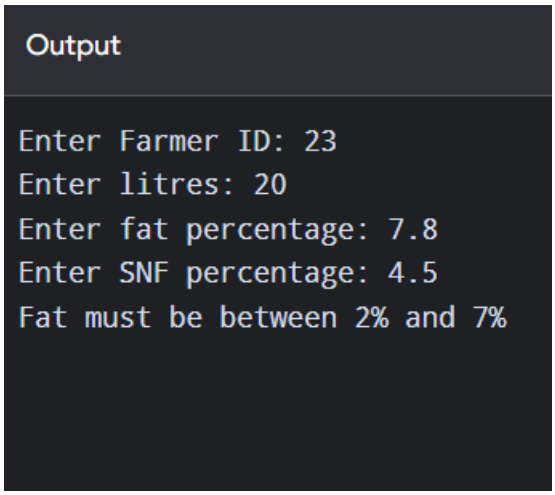
```
        snf = Decimal(input("Enter SNF percentage: "))

        if not farmer_id:
            print("Invalid Farmer ID")
        elif not litres.is_finite() or litres < 0:
            print("Invalid milk quantity")
        elif not fat.is_finite() or not MIN_FAT <= fat <= MAX_FAT:
            print("Fat must be between 2% and 7%")
        elif not snf.is_finite() or snf < 0:
            print("Invalid SNF percentage")
        else:
            rate = rate_per_litre(float(fat), float(snf))
            amount = payout(litres, fat, snf)

            print("\n--- PAYMENT DETAILS ---")
            print("Farmer ID:", farmer_id)
            print(f"Rate per litre: Rs. {rate}")
            print(f"Payment amount: Rs. {amount}")

    except (ValueError, ArithmeticError):
        print("Invalid input. Enter valid numeric values.")


if __name__ == "__main__":
    main()
```



Task 12:

Ans

A. Impact Analysis (2 Marks)

The new requirements introduce three major changes:

1. **Seasonal rates:** From April to June, the base rate changes to ₹32.00 and the fat step value changes to ₹0.45.
2. **Bonus payment:** Farmers supplying more than 500 litres in ten days receive an additional ₹0.50 per litre.
3. **Two-decimal readings:** The milk analyser provides fat and SNF values to two decimal places, so step calculations must handle decimal precision carefully.

The constants BASE_RATE, FAT_STEP_VALUE, and SNF_STEP_VALUE must be updated or selected according to the season. The functions rate_per_litre() and payout() must also be modified, and a new function is needed to calculate the ten-day bonus.

Since the analyser provides two-decimal readings, using Decimal for input and rate calculations improves precision. The number of 0.1% steps should be rounded to the nearest whole step.

B. Evaluation of the Original Design (2 Marks)

The original design uses fixed rate constants and calculates payments using fat and SNF steps. It is suitable for the original requirements but does not automatically support seasonal rates or bonus payments.

For example, consider milk with fat = 4.20% and SNF = 8.70%.

Fat steps:

$$\frac{4.20 - 3.50}{0.10} = 7$$

SNF steps:

$$\frac{8.70 - 8.50}{0.10} = 2$$

Under the original rate card:

$$\text{Rate} = 30 + (7 \times 0.40) + (2 \times 0.30)$$
$$\boxed{\text{Rate} = ₹33.40}$$

Under the April–June rate card:

$$\text{Rate} = 32 + (7 \times 0.45) + (2 \times 0.30)$$
$$\boxed{\text{Rate} = ₹35.75}$$

Therefore, the original design needs modification to select the correct seasonal rate and apply the bonus when eligible.

Program:

```
from decimal import Decimal, ROUND_HALF_UP


ORIGINAL_RATE_CARD = {
    "base": Decimal("30.00"),
    "fat_step": Decimal("0.40"),
    "snf_step": Decimal("0.30")
}


SEASONAL_RATE_CARD = {
    "base": Decimal("32.00"),
    "fat_step": Decimal("0.45"),
    "snf_step": Decimal("0.30")
}


MIN_FAT = Decimal("2.00")
MAX_FAT = Decimal("7.00")
BONUS_LIMIT = Decimal("500")
BONUS_RATE = Decimal("0.50")
STEP_SIZE = Decimal("0.10")
```

```
CENT = Decimal("0.01")


def select_rate_card(month):
    if month in (4, 5, 6):
        return SEASONAL_RATE_CARD
    return ORIGINAL_RATE_CARD


def rate_per_litre(fat, snf, rate_card):
    fat_steps = int(
        ((fat - Decimal("3.50")) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
    )

    snf_steps = int(
        ((snf - Decimal("8.50")) / STEP_SIZE).quantize(
            Decimal("1"), rounding=ROUND_HALF_UP
        )
    )

    rate = (
        rate_card["base"]
        + fat_steps * rate_card["fat_step"]
        + snf_steps * rate_card["snf_step"]
    )

    return rate.quantize(CENT, rounding=ROUND_HALF_UP)
```

```
def payout(litres, fat, snf, rate_card):
    rate = rate_per_litre(fat, snf, rate_card)
    amount = litres * rate
    return amount.quantize(CENT, rounding=ROUND_HALF_UP)



def bonus_amount(total_litres):
    if total_litres > BONUS_LIMIT:
        return (total_litres * BONUS_RATE).quantize(
            CENT, rounding=ROUND_HALF_UP
        )
    return Decimal("0.00")



def main():
    farmer_id = input("Enter Farmer ID: ").strip()
    month = int(input("Enter month number (1-12): "))

    if not farmer_id or not 1 <= month <= 12:
        print("Invalid Farmer ID or month")
        return

    rate_card = select_rate_card(month)
    total_litres = Decimal("0.00")
    total_payment = Decimal("0.00")

    for day in range(1, 11):
        print(f"\nDay {day}")
```

```
try:
    litres = Decimal(input("Enter litres: "))
    fat = Decimal(input("Enter fat percentage: "))
    snf = Decimal(input("Enter SNF percentage: "))

    if not litres.is_finite() or litres < 0:
        print("Invalid litres")
        continue

    if not fat.is_finite() or not MIN_FAT <= fat <= MAX_FAT:
        print("Fat must be between 2% and 7%")
        continue

    if not snf.is_finite() or snf < 0:
        print("Invalid SNF")
        continue

    amount = payout(litres, fat, snf, rate_card)
    rate = rate_per_litre(fat, snf, rate_card)

    total_litres += litres
    total_payment += amount

    print(f"Rate per litre: Rs. {rate:.2f}")
    print(f"Payment: Rs. {amount:.2f}")

except Exception as error:
    if isinstance(error, (ValueError, ArithmeticError)):
        print("Invalid numeric input")
    else:
```

```
                raise

    bonus = bonus_amount(total_litres)
    final_total = total_payment + bonus

    print("\n--- TEN-DAY PAYMENT REPORT ---")
    print("Farmer ID:", farmer_id)
    print(f"Total litres: {total_litres:.2f}")
    print(f"Milk payment: Rs. {total_payment:.2f}")
    print(f"Bonus: Rs. {bonus:.2f}")
    print(f"Final payment: Rs. {final_total:.2f}")


if __name___== "__main__":
    main()
```

OUTPUT:

```
Output

Enter Farmer ID: 23
Enter month number (1-12): 2

Day 1
Enter litres: 45
Enter fat percentage: 34.4
Enter SNF percentage: 9.5
Fat must be between 2% and 7%

Day 2
Enter litres: 10
Enter fat percentage: 3.4
Enter SNF percentage: 2.1
Rate per litre: Rs. 10.40
Payment: Rs. 104.00

Day 3
Enter litres: 4
Enter fat percentage: 2.1
Enter SNF percentage: 2.3
Rate per litre: Rs. 5.80
Payment: Rs. 23.20

Day 4
Enter litres:
```

D. Evidence-Based Defence (2 Marks)

The revised design uses separate rate cards for the original and seasonal rates. This makes rate revisions easier and avoids changing the calculation function whenever the rates change.

Test Results

For fat = 4.20%, SNF = 8.70%, and 10 litres:

Test	Rate per Litre	Milk Payment	Bonus	Final Payment
Original rates	₹33.40	₹334.00	₹0.00*	₹334.00*
April–June rates	₹35.75	₹357.50	₹0.00*	₹357.50*

*The table assumes the farmer's ten-day total is 10 litres, so the bonus does not apply.

For a farmer supplying 520 litres over ten days:

$$\text{Bonus} = 520 \times ₹0.50 = ₹260$$

The bonus is awarded because the total quantity exceeds 500 litres. A farmer supplying exactly 500 litres does not qualify because the rule states **more than 500 litres**.